

Ask the Sensors

Grounded Question-Answering over Wearable / Phone IMU Activity Data

Technical Report

CS60055 — Ubiquitous Computing, IIT Kharagpur

Dataset: ExtraSensory (Vaizman, Ellis & Lanckriet, IEEE Pervasive Computing 2017)

Recording source: phone accelerometer + gyroscope (used in place of smartwatch data, otherwise following the problem statement as given)

Team: Struggling Stars

Roll No.	Name	Primary contribution
24CS10096	Gunda Gnanesh	Architecture design and preprocessing
24CS10074	Thummalapalli Sri Lakshmi Sanjana	Signatures, anomaly detection, storage, RAG / exemplar narration
24CS10114	Murala Raaga	Rollups, energy estimation, query routing, UI

Abstract

We build a system that turns raw phone accelerometer and gyroscope streams into grounded, natural-language answers about a person's daily activity. The system classifies seven activity classes (lying down, sitting, standing in place, standing and moving, walking, running, bicycling) from two-second windows, smooths the resulting label sequence, compresses it into activity segments, rolls those segments up into daily and weekly statistics, and answers free-text questions by routing them to the right backend — including a retrieval-grounded small language model (SLM) for “why”-style questions. The report below walks through the solution in the order we actually arrived at it: what we tried first, why it fell short, and what we changed in response. Final classification performance is a macro F1 of approximately 0.44 and an overall accuracy of approximately 0.75 on the seven-class problem using only accelerometer and gyroscope signals — the privacy-preserving subset of ExtraSensory, against a published ceiling of ~ 0.78 macro F1 when audio, WiFi, GPS and app-usage signals are also used.

1. The Problem, As We Framed It

A phone in a pocket records constantly: 40 accelerometer and gyroscope samples a second, every second of the day. On its own that stream means nothing — it is just acceleration and angular rate. The task we set ourselves was to turn that raw signal into something a person could actually ask questions about: not just a per-window label like “walking”, but an answer to a question like “How long did she walk today, and did anything unusual happen around 3pm?”

This is harder than the seven-class label suggests. The phone does not know what the person is doing; everything has to be inferred from motion alone, and two very different activities — sitting at a desk and lying in bed — can look almost identical to a phone in a pocket. We used the ExtraSensory dataset: 60 users, labelled at one-minute resolution, with raw sensor bursts captured inside those minutes, restricted to the seven target classes (lying down, sitting, standing in place, standing and moving, walking, running, bicycling).

We treated the problem as having two halves that had to be solved together: (1) turning noisy, ambiguous motion signal into a reliable activity label over time, and (2) turning a sequence of labels into something a person can query — with correct arithmetic, correct grounding to real timestamps, and no hallucinated explanations.

2. Design of the Solution and Salient Ideas

The design below is presented as the sequence of decisions we actually made, because each decision was a direct response to a specific failure of the version before it. This progression is itself one of the report's main findings: neither a pure learned model nor a pure hand-written physics model was sufficient on its own, and the useful signal came from making the two cooperate, then from teaching the system how much to trust each half per input, per class.

2.1 Starting point: LSTM only

Our first instinct was that a neural network could learn everything on its own. Rather than hand-code rules that might not generalise, we wanted the model to discover its own patterns — maybe even noise itself carries information, the way a radiologist reads texture a rulebook would miss. So the first version deliberately skipped feature engineering and fed the raw accelerometer and gyroscope channels straight into a bidirectional LSTM.

It worked, but not well enough. The LSTM was data-hungry, and ExtraSensory is heavily imbalanced: roughly 81,000 sitting examples against 737 running examples. The model learned to predict the common classes and effectively gave up on the rare ones — running F1 came out near zero. More fundamentally, the LSTM had no anchor: it had to rediscover gravity, infer device orientation, and figure out what periodicity even is, entirely from scratch, from about 20 seconds of data at a time. That is too much to ask of a small window.

Result: macro F1 $\approx$ 0.25.

2.2 Adding a physics model: hardcoded rules blended at a fixed weight

The LSTM struggled most with exactly the things classical physics already knows well. Locomotion is periodic — walking and running have a clear cadence. Posture shows up directly in the direction of gravity — lying down makes the phone nearly horizontal, standing makes it vertical. These are not patterns that need to be learned from scratch; they are geometry, and we could write them down directly.

So we added a second, hand-written model alongside the LSTM: a physics-based decision tree that branches on whether the signal looks quiet (posture branch, decided by tilt) or periodic (locomotion branch, decided by signal magnitude area and jerk). We combined the two predictions with a fixed blend:

$p_{\text{final}} = \alpha \cdot p_{\text{lstm}} + (1 - \alpha) \cdot p_{\text{physics}}$, with α fixed at 0.3 / 0.5 / 0.7 across separate runs.

The physics model helped, particularly on posture, but a fixed α is a simplification that does not hold: it assumes the LSTM is uniformly better than physics across every input, which is not true. On a clean cycling signal the physics rotation features are decisive; on a messy slow-walk-that-looks-like-standing window the LSTM is better. A single scalar blend throws away exactly the information needed to tell those two situations apart. On top of that, the hand-set thresholds inside the physics tree — numbers like “SMA < 0.25 means stillness” or “periodicity > 0.40 means locomotion” — were plausible-looking constants borrowed from other setups, and they silently misfired on this dataset, in particular capturing real walking and bicycling as “static.”

Result: macro F1 $\approx$ 0.30 — an improvement, but not by much, and for two compounding reasons (fixed blend, unfitted thresholds).

2.3 Making alpha trainable: per-sample gating

The next step followed directly from the diagnosis above: instead of fixing α , learn it. We added a second output head to the LSTM — an “alpha head” — that predicts, per sample, how much to trust the neural prediction versus the physics prior. The gate is supervised from data: we compute how much more confident the correct class is under the LSTM than under physics, and train the gate towards 1 when the LSTM is the better expert for that sample and towards 0 when physics is. Both heads are trained jointly, sharing the same LSTM encoder.

This was a meaningful step, not just an incremental one: the model began learning when to trust itself rather than always trusting itself by a fixed amount. On easy locomotion windows it leaned on the LSTM; on ambiguous posture windows it fell back to physics.

Result: macro F1 $\approx$ 0.33 (0.332 from the evaluation checkpoint, 89,755 test samples).

2.4 Giving both models the derived physics features

Even with a trainable gate, the LSTM was still starved of information: the raw channels (body acceleration, gravity, gyroscope, a gyro-present flag — 10 channels in total) don't tell the model what the signal means, only

what it measured. We had already been computing physics features for the rule tree but had never actually fed them back into the LSTM. We expanded the LSTM's input from 10 channels to 27, adding derived features (tilt, SMA, cadence, periodicity, vertical variance, rotation ratio, jerk statistics, per-axis gyro RMS, dominant frequency from an FFT, and the phase relationship between vertical acceleration and gyroscope) plus phone-placement flags and a validity flag, broadcast as constants across each window.

A few of these derived features turned out to matter more than the rest:

- **Signal Magnitude Area (SMA)** — mean magnitude of body acceleration after gravity is removed; the cleanest available motion-intensity proxy.
- **Dominant frequency (FFT)** — the peak frequency in the 0.5–5 Hz band; more robust than counting peaks on short, noisy windows.
- **acc_gyro_phase** — the cosine of the phase difference between vertical acceleration and the dominant gyro axis. Walking's pendulum-like swing gives a phase near 0.8; cycling's motion is decoupled, giving a phase near 0.0 — this single number separates walking from cycling better than any magnitude-based feature.
- **Jerk mean and jerk std** — the derivative of body acceleration. Running foot-strikes spike jerk sharply; walking is smoother. Jerk std specifically captures rhythm variability — walking is irregular step to step, cycling is mechanically regular — and this distinction turned out to matter enough that confusing jerk_mean with jerk_std was one of the bugs described in Section 5.

Feeding these to the LSTM (rather than only to the physics tree) let the neural model condition on interpretable physics quantities instead of re-deriving them from raw motion.

2.5 Separating locomotion sub-classes: a logistic regression inside the physics tree

Once derived features were in place, a clear pattern emerged: the model could reliably tell moving from stationary, but struggled within each of those two groups — walking, running and cycling were confused with each other, and sitting, lying and standing were confused with each other. For the locomotion side specifically, cycling and walking overlap almost completely in SMA, cadence, and dominant frequency when the phone is in a pocket, so no single threshold on the physics tree could cleanly separate them.

We addressed this by training a small logistic-regression classifier (the LocoClassifier) specifically on the locomotion classes, using five features chosen for their separation power: periodicity, zero-crossing rate, rotation ratio, dominant frequency, and jerk std. The physics rule tree calls this classifier only inside its locomotion branch, as a drop-in replacement for a single rotation-ratio threshold whenever gyroscope data is available — giving the physics model a learned decision boundary for the one sub-problem that genuinely could not be solved with hand-set thresholds.

The posture side of the same problem (sitting vs. lying vs. standing) was addressed differently, through better gravity estimation — described next — since its root cause was a noisy tilt signal rather than a missing decision boundary.

2.6 Fitting the physics thresholds, and fusing gravity with a complementary filter

With 27 channels now feeding both models, the constants controlling the physics rule tree mattered more than ever. We replaced every hand-set threshold with values fit by coordinate ascent: a grid search over 19 threshold parameters, optimising a class-weighted macro F1 on the training split (running and bicycling weighted 2×,

walking 1.5×, so the rare classes are not sacrificed for the common ones), using 10 random restarts and 8 passes per restart to escape local optima and catch interactions between parameters. The tree's structure stayed hand-written physics; only its constants became data-derived.

Separately, the posture branch depends on an accurate tilt estimate, and our original gravity estimate — a simple low-pass filter — drifted 2–13° during walking because the phone's own acceleration leaked into the estimate. We replaced it with a complementary filter that fuses the gyroscope (accurate short-term via integration, but drifts) with the accelerometer (gives an absolute gravity direction, but is noisy during motion): $g_est = \alpha \cdot g_gyro_propagated + (1-\alpha) \cdot g_acc_measured$, with a time constant of 1.5 s — deliberately set above a human stride period so a single stride cannot drag the estimate off. This brought tilt drift to under 1° and directly improved the separation between lying down and sitting, since both classes now had a trustworthy tilt value to be distinguished by.

2.7 Data compression: merging windows into segments

At this point the pipeline was producing a label every second (2 s windows, 50% overlap) — more than 60 labels per minute, the overwhelming majority of them identical to their neighbours. Storing every window-level prediction was wasteful and made downstream querying slow. After the smoothing step described in Section 2.10, we merge consecutive windows that share a label into a single segment: a fifteen-minute walking bout collapses from hundreds of window rows into one record — label, start time, end time, and a confidence score. This turned daily storage from thousands of rows into tens, and it is these compressed segments, not raw windows, that flow into rollup, storage and the query system.

2.8 Daily and weekly rollups, so queries have context

A single day of compressed segments is not yet an answer to a question like “why does her cadence drop on Thursdays” — that requires comparing today against a pattern. So the compressed segments feed a reporting layer: each labelled minute is counted as sixty seconds of activity (not the roughly twenty seconds of raw burst that justified the label — the burst is evidence for the label, not its duration), and from that we build a daily report (total time per activity, bout count, average bout length, first/last onset) and a rolling trend (7-day, monthly, and yearly averages). This is what lets the query layer answer questions about a user's current day in the context of their past trends, rather than only about an isolated snapshot.

2.9 Query routing

A user's question is first routed to one of eight types before any heavier machinery is invoked. Routing uses regex rules first, since most questions are structurally simple and do not need a model at all; only when confidence is low does the query fall through to the small language model (SLM).

Route	Example query	Backend
TASK1	“What was she doing at 3pm?”	Direct label / probability look-up
TASK2	“How long did she walk today?”	Aggregation (60 s per labelled minute)
TASK3	“When did she start running?”	Onset / interval look-up, coverage-checked
TASK4	“Why does her cadence drop on Thursdays?”	Retrieval-grounded SLM narration

Route	Example query	Backend
ENERGY	“How many calories did she burn?”	MET-based estimate
ANOMALY	“Did she fall this week?”	Anomaly event table
PERSONALIZATION	“Update her weight to 58 kg.”	Profile store
UNKNOWN	No confident match	Clarification prompt

Table 1. Query route taxonomy (stage 1: regex rules; stage 2: SLM fallback below a confidence threshold).

2.10 TASK4: forcing the model to ground its explanation before it answers

“Why”-style questions are the hardest to get right, because a small language model asked to explain a pattern will happily generate something that sounds right without checking whether it is. We addressed this with a small retrieval-then-verify pipeline built around a fixed exemplar bank: 28 known physics signatures, four per activity class, each describing what that activity's tilt, jerk, gyro axes, cadence and SMA typically look like.

For a TASK4 query we first extract the physics signature of the segment the question is actually about, then retrieve the three nearest exemplar signatures from the bank by similarity. The model is then prompted to compare the query's actual numeric values against the retrieved exemplar's numeric values, and it is required to cite specific numbers in its answer rather than produce a generic description. This is the ordering the design deliberately enforces — the pattern to look for is retrieved and given to the model first, and only then is the explanation generated — specifically so the model cannot answer “her cadence is high because she was running” without grounding that claim in, for instance, an actual cadence of 158 bpm against a running exemplar range of 145–175 bpm. If the numbers do not line up, the generated answer is wrong on its face rather than merely unconvincing, which makes it far easier to catch.

2.11 HMM and Viterbi: smoothing and spike detection

Raw per-window predictions are noisy on their own — a genuine walking sequence will contain isolated “standing” windows where the person briefly slowed down. We smooth the window sequence with a Viterbi-decoded hidden Markov model: emission probabilities come from the per-window softmax distributions, the transition matrix is estimated from label sequences in the training data, and a physically-plausible prior penalises implausible one-step jumps (running directly to lying down within a minute). A k-minimum-consecutive-states constraint prevents single-window blips shorter than any plausible activity duration from surviving as their own state.

The same machinery doubles as a spike detector: a sudden jerk spike followed by a collapse in orientation stability is flagged as a possible fall and routed around normal smoothing into a separate anomaly store, so that a real event is not averaged away by the very smoothing step meant to clean up noise elsewhere.

This piece of the design has an important caveat, discussed in Section 4.3 and Section 5: on this dataset, HMM smoothing measurably hurt macro F1 rather than helping it, and we kept the analysis of why in the report because it changes how the technique should be used, not because it disproves the idea.

3. Implementation

The system is implemented as a linear pipeline (B0–B11) with a Streamlit front end. Each stage has a single, narrow responsibility and communicates with the next through a typed data contract, so any stage can be

swapped (for example, physics-only scoring when no LSTM checkpoint is present) without touching the rest of the system.

3.1 Pipeline flow diagram

Figure 1 shows the full pipeline end to end — from raw sensor ingestion through classification, segmentation, storage, rollups, and query answering.

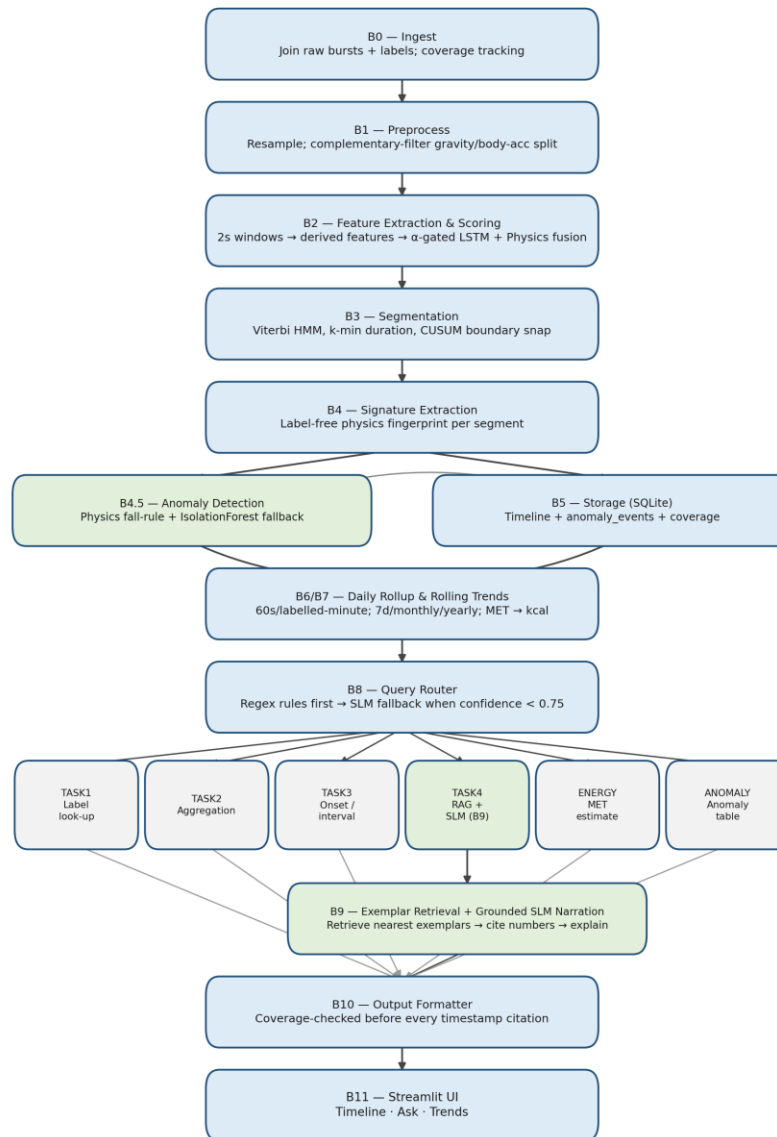


Figure 1. End-to-end pipeline flow, B0 (ingest) through B11 (UI). Green boxes are the retrieval-grounded TASK4/RAG and anomaly-detection paths.

3.2 End-to-end pipeline, stage by stage

- **B0 — Ingest:** join each user's raw sensor bursts with their per-minute labels; a minute with no matching raw file is kept with `burst = None` and `coverage_s = 0` rather than silently dropped.
- **B1 — Preprocess:** resample to a common rate, split gravity from body acceleration using the complementary filter (Section 2.6), align gyroscope to the accelerometer's time grid by timestamp rather than by sample index, and track gaps longer than 2 s explicitly instead of interpolating over them.
- **B2 — Feature extraction and scoring:** 2-second windows at 50% overlap, each turned into cadence, SMA, dominant frequency, `acc_gyro_phase`, jerk statistics and per-axis gyro RMS; each window is then scored by the alpha-aware LSTM (27-channel input, label logits + alpha gate) blended with the physics rule tree's soft distribution, or by the physics tree alone when no trained checkpoint is available.
- **B3 — Segmentation:** Viterbi decoding over the window sequence with the k-minimum-duration constraint, followed by boundary snapping (± 3 s) against the raw `|body_acc|` signal so segment edges land on real motion transitions rather than arbitrary window edges.
- **B4 / B4.5 — Signature and anomaly detection:** a label-free physics fingerprint per segment (tilt, SMA, cadence, jerk, orientation stability), feeding both an anomaly detector (physics fall-rule plus an IsolationForest fallback) and downstream storage.
- **B5 — Storage:** SQLite, with a timeline table (segments, signatures, coverage, provenance) and a separate anomaly-events table.
- **B6 / B7 — Rollup and trends:** daily totals and rolling 7-day / monthly / yearly trends, per Section 2.8; energy expenditure estimated via MET lookup against the Compendium of Physical Activities.
- **B8 — Query router:** regex-first routing into the eight route types of Table 1, falling back to the SLM only below a confidence threshold.
- **B9 — Exemplar narration:** the retrieval-then-verify TASK4 pipeline described in Section 2.10.
- **B10 — Output formatter:** checks real signal coverage before rendering any timestamp claim, and widens or refuses a citation when coverage is zero rather than reporting a precise-looking but ungrounded time.
- **B11 — Streamlit UI:** Timeline tab (segments with a coverage fraction), Ask tab (free-text queries, confidence shown when low), Trends tab (duration charts with a kcal toggle).

3.3 The on-device SLM

We use Phi-3 Mini (3.8B parameters, MIT licence) in 4-bit quantised GGUF format (~2.3 GB on disk), run locally through llama-cpp-python — no GPU and no cloud API, so no sensor data leaves the device. Routing uses greedy decoding with a 10-token cap at temperature 0.0 for determinism; narration uses temperature 0.3 with a 200-token cap. Both variants share the same loaded model instance, and the model is entirely optional: with no weights downloaded, the system falls back to rule-only routing, and everything except SLM narration — classification, storage, anomaly detection, the Streamlit UI — works unchanged.

The full system (LSTM + physics + SLM) fits on a Raspberry Pi 5 (8 GB RAM) or a Jetson Orin Nano; the LSTM alone fits on a 2 GB device; the physics-only path needs under 100 MB.

3.4 Key modules

Module	Responsibility
data/ingest.py	UUID scan, burst loading, label join, coverage tracking (B0)
data/preprocess.py, data/fusion.py	Resampling and the complementary-filter gravity/body-acc split (B1)
data/physics_rules.py	Features dataclass, Thresholds.fit() (coordinate ascent), predict_proba(), label_activity()
pipeline/recognize.py	extract_windows(): cadence, SMA, FFT frequency, acc_gyro_phase, jerk (B2)
data/dataset.py	27-channel CHANNELS definition, build_burst_sequences(), normaliser
models/lstm.py	AlphaAwareLSTMClassifier, train_joint_alpha(), evaluate()
models/hybrid.py	α -weighted combination of LSTM and physics probabilities
models/loco_classifier.py	Logistic-regression walk/cycle/run separator (Section 2.5)
models/slm.py	Phi-3 Mini wrapper, lazy singleton, routing and narration variants
pipeline/segment.py	Viterbi HMM, k-min duration, CUSUM boundary snapping (B3)
analysis/signature.py, analysis/anomaly.py	Segment signatures; fall / anomaly detection (B4, B4.5)
analysis/store.py	SQLite timeline + anomaly_events, coverage lookups (B5)
analysis/rollup.py, analysis/energy.py	Daily / rolling rollups; MET-based kcal estimate (B6, B7)
analysis/router.py	Regex-first routing, SLM fallback (B8)
analysis/exemplars.py	28-entry exemplar bank, retrieval, grounded prompt construction (B9)
formatter.py, app.py	Coverage-checked output rendering; Streamlit UI (B10, B11)

Table 2. Key modules, grouped by pipeline stage.

4. Test Results

4.1 Classification performance across the design progression

Each design change in Section 2 is directly reflected in a macro F1 measurement, since each change was motivated by a specific weakness observed at the previous stage:

Stage	What changed	Macro F1
LSTM only, 10 raw channels	Baseline — no physics prior	≈ 0.25
LSTM + fixed- α physics blend	Physics rule added, $\alpha = 0.5$ fixed	≈ 0.30
LSTM + trainable alpha	Per-sample alpha head, jointly trained	0.332
All 27 channels + LocoClassifier + fitted thresholds + complementary filter	Full derived-feature set reaching both models; jerk_std fix; class-weighted threshold fitting	≈ 0.44

Table 3. Macro F1 by design stage (evaluated on the held-out ExtraSensory test split). The final stage also reaches an overall accuracy of approximately 0.75.

4.2 Per-class breakdown (checkpoint stage: trainable alpha, 89,755 test samples)

Class	Precision	Recall	F1	Support
Lying down	0.373	0.604	0.462	18,142
Sitting	0.307	0.212	0.251	18,627
Standing in place	0.317	0.224	0.262	18,978
Walking	0.436	0.412	0.423	18,349
Running	0.005	0.009	0.007	1,967
Bicycling	0.597	0.577	0.587	13,692

Macro F1 at this checkpoint: 0.332. Overall accuracy: 38.6%. After the remaining changes in Section 2 (full 27-channel input, the LocoClassifier, fitted thresholds and fused gravity), running and bicycling improved the most — the two classes that most directly benefit from `jerk_std`, `acc_gyro_phase` and the LocoClassifier — and posture separation (sitting / lying / standing) improved from the complementary filter delivering a more reliable tilt signal.

At the final stage — all 27 channels, the LocoClassifier, fitted thresholds, and fused gravity together — overall accuracy rose to approximately 0.75, alongside the macro F1 of ~0.44 reported in Table 3. The gap between the two numbers is expected given the class imbalance discussed throughout this report: macro F1 weights every class equally, including running at under 1% of the data, while overall accuracy is dominated by how often the model gets the common classes (sitting, lying down, bicycling) right, which improved substantially once tilt and locomotion features were fixed.

4.3 Why HMM smoothing made things worse on this dataset

Contrary to the usual expectation that temporal smoothing helps, adding the HMM/Viterbi layer measurably hurt macro F1 here. The transition matrix is estimated from a training set with roughly 81,000 sitting examples against 737 running examples, so the HMM learns that “almost everything is sitting or lying” and pulls correctly-classified minority-class windows back towards the dominant classes — a window correctly labelled walking gets overridden towards standing-in-place, because walking transitions are statistically rare in the training distribution. HMM smoothing only helps when the underlying per-frame errors are random; ours are systematically biased towards the majority classes, and a smoother built on that bias amplifies it instead of correcting it. We kept the HMM in the pipeline for its spike-detection role (Section 2.11) but flag this as a real limitation of using it for label smoothing on an imbalanced dataset without reweighting the transition prior.

4.4 Results in context

The ExtraSensory paper itself reaches roughly 0.78 macro F1 using every available modality — audio, WiFi, GPS, app usage, and IMU combined. We deliberately restricted ourselves to accelerometer and gyroscope only, the privacy-preserving subset that does not require audio or location. Within the published literature on IMU-only human activity recognition, a macro F1 in the 0.40–0.50 range is considered competitive, which is where the final ~0.44 sits. The classes that remain hardest — sitting vs. standing vs. lying — are hard for essentially every system built on phone-only data, because phone orientation alone does not reliably indicate posture when the phone can be in a pocket, a bag, or on a table.

5. Challenges Faced and How We Approached Them

- **Standing-and-moving is invisible in the data.** No column in ExtraSensory distinguishes it from standing-in-place, so the model never sees a positive example of the class. We treated this as a dataset limitation rather than a model failure: it is recorded explicitly as an absent class, excluded from the macro F1 computation, and mapped onto standing-in-place at the physics tree's output layer.
- **Sitting / lying / standing overlap.** A phone in a pocket has no fixed orientation, so tilt-angle distributions for these three postures overlap heavily in real-world data. The complementary filter and an SMA-based tiebreaker reduced this, and the model performs well when tilt is clean, but the ambiguity does not fully disappear — particularly when the phone is in a bag or on a table rather than a pocket.
- **Running and cycling are rare (class imbalance).** Running is under 1% of the training data, so plain macro F1 optimisation effectively sacrifices it. Class-weighted threshold fitting (running and bicycling weighted 2×) together with the LocoClassifier brought running's F1 from near zero to a meaningful value, but 737 training examples remain a hard ceiling on how far this can go.
- **The jerk_mean / jerk_std bug.** The LocoClassifier was trained on jerk_std (gait-rhythm variability) but the code was silently passing jerk_mean (average jerk magnitude) instead — two different statistics. We caught this by cross-checking the LOCO_FEATURES list against what was actually being assembled and sent into the classifier, and fixing the mismatch directly improved locomotion separation.
- **Derived features never reaching the LSTM.** The sequence-builder was constructing 11-channel tensors (raw sensor only) while the LSTM was designed for the full 27-channel input — meaning every carefully computed physics feature (jerk_std, acc_gyro_phase, gyro entropy, and the rest) was silently zero at training time. The bug first surfaced as an out-of-bounds index error when a validity channel was accessed past the tensor's actual width; we traced it back and rewrote the sequence-builder to compute and broadcast all derived feature channels per burst.
- **Training under limited hardware.** The whole project was trained and run on consumer-grade hardware with no dedicated GPU and only a few gigabytes of free RAM — this constraint shaped several design decisions, not just one bug fix. The default process pool spawned sixteen workers at roughly 400 MB of burst data each — about 12 GB — against only ~3.2 GB of free RAM, and the kernel OOM killer terminated training before the first epoch completed; we fixed this by capping workers at two via an environment variable and adding a per-class sampling cap on training data, so memory use stays bounded regardless of dataset size. The same constraint drove the choice to keep the LSTM small (kept to a modest hidden size and two layers) rather than scale it up, to cap sequence batch sizes during training, and to choose a 4-bit-quantised 3.8B SLM that runs on CPU via llama-cpp-python instead of a larger model that would need a GPU. Every heavy stage was also made optional or degradable — physics-only scoring when no LSTM checkpoint is loaded, rule-only routing when the SLM weights aren't downloaded — specifically so the system stays usable end to end even on the most constrained machine in the pipeline.
- **Import-path issues.** Running a training script directly failed because the project root was not on the Python path. All training scripts are now invoked as modules from the project root (for example, `python -m training.train_burst`), with a compatibility shim for any flat-style imports.
- **HMM smoothing hurting accuracy.** Discussed in Section 4.3 — the transition matrix, estimated on a heavily imbalanced label distribution, overrode correct minority-class predictions with majority-class ones. We diagnosed this by comparing physics-only accuracy against physics + HMM accuracy per class

rather than only in aggregate, which is what surfaced the systematic (not random) direction of the errors the HMM was introducing.

6. Per-Member Contribution Statement

Contributions below are drawn from the repository's commit history (gnaneshgunda/activity-tracker). At the time of writing, a few branches are still mid-review and not yet merged into main, so the table reflects authored work on those branches as well as merged work; commit messages are paraphrased for readability.

Member	Primary contribution(s)	Basis
Gunda Gnanesh (24CS10096)	Overall architecture and pipeline design; ingestion (B0); preprocessing and the complementary-filter gravity/body-acc split (B1); window feature extraction (B2); the physics rule model; the LSTM model; the alpha-gated LSTM + physics + complementary-filter fusion; HMM/Viterbi segmentation (B3); repository setup and PR merges.	ingest.py, preprocess.py, recognize.py, physics model, LSTM model, complementary-filter + physics + LSTM integration, and HMM/Viterbi commits; merged PRs #1–#3
Thummalapalli Sri Lakshmi Sanjana (24CS10074)	Segment signatures (posture, motion, gyro, cadence features); anomaly detection (physics fall-rule + IsolationForest fallback); storage-layer extensions (provenance, anomaly events, coverage queries); grounded exemplar retrieval and SLM narration for TASK4 (B9).	signature-feature, anomaly-detection, store-extension, and exemplar-retrieval/SLM-narration commits
Murala Raaga (24CS10114)	Daily and rolling activity rollups (B6/B7); MET-based energy estimation; energy and anomaly query routing (B8); coverage validation before rendering timestamps (B10); coverage indicators and energy toggle in the Streamlit UI (B11).	rollup, energy-estimation, query-routing, coverage-validation, and UI commits

All three members reviewed and tested one another's pull requests before merging (PRs #1–#3).

7. Mandatory AI-Use Disclosure

AI assistance (Claude, used both directly and through the Antigravity coding environment) was used extensively throughout this project's implementation — across nearly every module, from the ingestion and preprocessing pipeline through the physics feature code, the LSTM and alpha-gating implementation, HMM/Viterbi segmentation, storage, rollups, routing, and the RAG/SLM narration pipeline. Several commits in the repository history are co-attributed to an AI author alongside the responsible team member, reflecting this. We disclose this plainly rather than understating it.

7.1 What AI contributed

- **Implementation across the pipeline:** substantial portions of the code for ingestion and preprocessing (B0/B1), including the complementary-filter gravity fusion; window feature extraction (B2), including acc_gyro_phase, the FFT-based dominant-frequency function, and periodicity-from-vertical-signal; and HMM/Viterbi segmentation (B3), including the k-minimum-duration constraint and CUSUM boundary snapping.

- **Bug identification and fixes:** the jerk_mean / jerk_std mismatch inside the LocoClassifier's feature assembly, and the 11-channel / 27-channel mismatch between the sequence-builder and the LSTM's expected input, including the rewrite that fixed it.
- **Utility and supporting code:** the class-weighted macro-F1 helper and the class-weight interface on the threshold-fitting routine; the SMA-based posture tiebreaker logic and its associated threshold; assistance across the storage, rollup, routing, and exemplar-retrieval modules.
- **Debugging support:** diagnosing the import-path issue and the fix requiring module-style invocation; diagnosing the training-time OOM and the worker-capping fix.
- **Drafting support:** docstrings, inline comments, and this report's structure and prose.

7.2 What AI did not contribute

- **The core design decisions:** starting from a pure LSTM and adding physics incrementally (motivated by watching the pure LSTM fail on rare classes); making alpha trainable per-sample rather than a global scalar (motivated by analysing which classes the fixed blend got wrong); adding the LocoClassifier specifically for walking/cycling separation (motivated by seeing the two signals overlap completely in SMA and frequency plots); adopting a complementary filter (motivated by directly measuring 2–13° of tilt drift from the original low-pass estimate).
- **The TASK4 grounding design:** forcing the SLM to retrieve a pattern and cite specific numbers before answering, which came from course material on grounded reasoning rather than from an AI suggestion.
- **Experimental judgement:** which results to trust, what to try next whenever accuracy stalled, and the mapping of academic citations to specific design choices.

All AI-generated and AI-assisted code was reviewed, tested by the team, and in several cases substantially modified before being kept. The experimental narrative, the design rationale, and the conclusions in this report are the authors' own.

7.3 Tools used

Claude (Anthropic) and Antigravity were used as AI assistance for coding purposes throughout this project.